A Predictive Learning Framework for Monitoring Aggregated Performance Indicators over Business Process Events

Alfredo Cuzzocrea
Dept. DIA, University of Trieste
Trieste, Italy
alfredo.cuzzocrea@dia.units.it

Francesco Folino, Massimo Guarascio, and Luigi Pontieri
Institute ICAR, National Research Council (CNR)
Rende, Italy
name.surname@icar.cnr.it

ABSTRACT

In many application contexts, a business process' executions are subject to performance constraints expressed in an aggregated form, usually over predefined time windows, and detecting a likely violation to such a constraint in advance could help undertake corrective measures for preventing it. This paper illustrates a prediction-aware event processing framework that addresses the problem of estimating whether the process instances of a given (unfinished) window w will violate an aggregate performance constraint, based on the continuous learning and application of an ensemble of models, capable each of making and integrating two kinds of predictions: single-instance predictions concerning the ongoing process instances of w, and time-series predictions concerning the "future" process instances of w (i.e. those that have not started yet, but will start by the end of w). Notably, the framework can continuously update the ensemble, fully exploiting the raw event data produced by the process under monitoring, suitably lifted to an adequate level of abstraction. The framework has been validated against historical event data coming from real-life business processes, showing promising results in terms of both accuracy and efficiency.

CCS CONCEPTS

• **Computing methodologies** → **Supervised learning by regression**; **Online learning settings**; • **Applied computing** → **Business process monitoring**; **Business intelligence**; **Event-driven architectures**;

KEYWORDS

Business Process Performance, Business Process Intelligence, Data Streams

ACM Reference Format:
Alfredo Cuzzocrea and Francesco Folino, Massimo Guarascio, and Luigi Pontieri. 2018. A Predictive Learning Framework for Monitoring Aggregated Performance Indicators over Business Process Events. In *IDEAS 2018: 22nd International Database Engineering & Applications Symposium, June 18–20, 2018, Villa San Giovanni, Italy*. ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3216122.3216143

IDEAS 2018, June 18–20, 2018, Villa San Giovanni, Italy

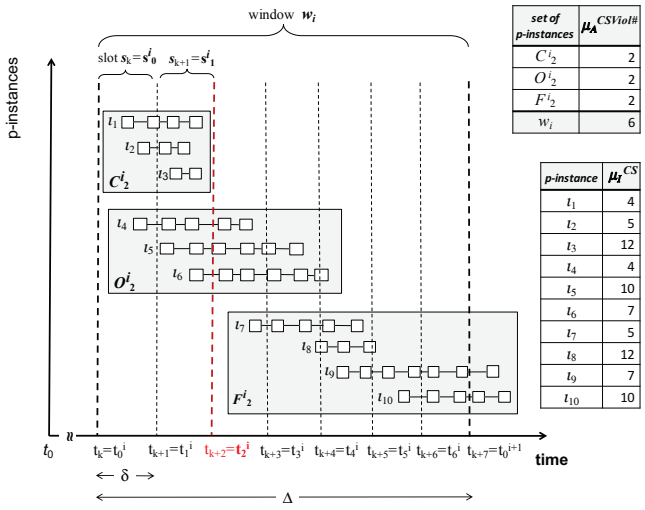
ACM ISBN 978-1-4503-6527-7/18/06.
https://doi.org/10.1145/3216122.3216143

1 INTRODUCTION

A *Process Performance Indicator* (PPI) [4] is a special form of KPI that is meant to support the evaluation of performance measures and associated constraints (concerning, e.g., process duration/costs) over a business process' executions. PPIs can be of two kinds [2]: *single-instance PPI* (*S-PPI*) and *aggregate PPI* (*A-PPI*), defined at the levels of single process instances (*p-instance*) and of process instances' sets, respectively. The latter kind is usually applied to sets of p-instances, named hereinafter "process windows" (or simply windows), resulting from segmenting the event stream produced by a process using fixed-length non overlapping time window.

Example 1. Let P_{ex} be a ticket-management process, carried out in a service center for a client company. Each p-instance, say ι, of this process corresponds to the handling of a ticket (arisen from a client's help request), and it is eventually assigned a *Customer Satisfaction (CS) level*, denoted as $\mu_I^{CS}(\iota)$ and representing a sort of performance measurement. Suppose that two performance constraints exist for the process: *(C1)* "the CS level of each ticket cannot be lower than 8 (over a 1-12 scale)"; *(C2)* "at most 5 tickets per each week can have a CS level lower than 8". The former constraint can be checked on each p-instance ι by using some suitable S-PPI $\phi^{CS \geq 8}$ focusing on the single-instance performance value $\mu_I^{CS}(\iota)$; the latter constraint can be checked on any set w of p-instances started in the same week, i.e. a (process) window of duration 1 week, by using some A-PPI $\Phi^{CSViol\# \leq 5}$ that focuses on an aggregate performance value $\mu_A^{CSViol\#}(w)$ computed as the number of instances of w that violate $\phi^{CS \geq 8}$. Figure 1 shows an example window w_i consisting of 10 p-instances, the performance value of each p-instance (computed with μ_I^{CS}), and the aggregate performance values (computed with $\mu_I^{CSViol\#}$) of w_i and of three w_i's subsets. □

PPIs provide a conceptual basis for monitoring the performances of a process, and quantitatively assessing the alignment of a business process to performance constraints, based on the stream of event data that the process generates. Indeed they can combined with BAM [1, 3] and CEP [11, 17] technologies to detect violations to these constraints. However, BAM/CEP solutions usually lack process-aware predictive monitoring capabilities and they are mainly devoted to help react to detected deviations. In general, "Predictive Process Monitoring" approaches [13–15, 18] try to extend process monitoring infrastructures with such prediction capabilities. However, most of these approaches rely on applying ad hoc learning methods [6, 8, 15, 16, 19]) to historical log data to induce a *Predictive Process Model (PPM)* that can predict the performance outcome of a p-instance p (while it is still running), based on its partial *trace* [18] —i.e. the sequence of log events available for p up to



set of p-instances	$\mu_A^{CSViol\#}$
C^i_2	2
O^i_2	2
F^i_2	2
w_i	6

p-instance	μ_I^{CS}
ι_1	4
ι_2	5
ι_3	12
ι_4	4
ι_5	10
ι_6	7
ι_7	5
ι_8	12
ι_9	7
ι_{10}	10

Figure 1: A process window (w_i) for an example ticket management process, of duration $\Delta = 1$ week and consisting of 10 p-instances ($\iota_1, \ldots, \iota_{10}$). The tables reports performance measurements for each p-instances and for some p-instance sub-sets, defined w.r.t. the prediction time ("checkpoint" t^i_2).

the prediction time (named hereinafter a *checkpoint*). Hence, such predictive monitoring approaches tell nothing on the performances of p-instances that will start after the current checkpoint.

The works in [2, 9] tried to bring the perspective of predictive process monitoring to the level of aggregate PPIs, facing the following run-time prediction problem: given an A-PPI Φ for a process P, predict whether an ongoing window w_i of P will eventually violate the constraint associated with Φ, at different checkpoints (like $t^i_0, \ldots, t^i_6$ in Figure 1) as far as w_i unfolds. In general, when trying to predict at time t^i_j whether w_i will be violating, only the performance outcomes of the p-instances of w_i that finished before t^i_j are known, while the compliance of w_i to Φ also depends on the behavior of two further subsets of w_i: the set O^i_j of "ongoing" instances still running at time t^i_j; and the set F^i_j of "future" instances that will start after t^i_j. These sets are shown in Figure 1 for checkpoint t^i_2, in a scenario where the monitoring is performed using the A-PPI $\Phi_A^{CSViol\#\leq 5}$ of Example 1 —notice that the window in the figure is positive, as it contains 6 p-instances with a CS level lower than 8.

The solutions proposed in [2, 9] allow for inducing a window-oriented kind of prediction model, named *APPM*, capable of making and integrating forecasts for both the ongoing and future p-instances. However, both works overlooked the key practical problem of embedding the discovery and application of such a window-oriented predictor into an online event processing framework, capable to update both the prediction for the current process window and the prediction model itself on the basis of streaming event data.

To overcome the limitations of state-of-the-art predictive monitoring solutions, a prediction-aware event processing framework is proposed here that extends the core learning methods of [2, 9] with event-processing and online learning capabilities. The framework (described in Sec. 4) relies on dynamically learning and applying an ensemble of APPMs by processing and transforming (through suitable aggregation and abstraction mechanisms) the event data that the process under monitoring produces. Technically, the framework leverages a customised version of the learning scheme proposed in [5] to incrementally train an ensemble of APPMs from chunks of the event stream, and to return an overall prediction for the current window and evaluation time, computed by combining the predictions of all the APPMs via a time-adjusted voting scheme. Notably, this allows us to work effectively in non-stationary process monitoring environments and to inherit the ability of ensemble-based prediction approaches to provide stable results [10].

Tests performed against the logs of two real-life business processes confirmed the validity of our approach in terms of both accuracy and efficiency (see Section 5). The next two sections, namely Sections 2 and 3, are devoted to introduce some preliminary concepts and the problem setting, respectively.

2 PRELIMINARIES

Single-instance metrics and PPIs. A *p-instance metric* for P is a function $\mu_I : \mathcal{I}_P \rightarrow \mathbb{R}_{\geq 0}$ that maps any p-instance ι of P to a performance value. W.l.o.g., let us focus on the case where μ_I is an efficiency score (e.g., the time spent to carry out ι), such that the lower $\mu_I(\iota)$, the better the performance outcome of ι.

To reason on deviations of a p-instance metric's result from some existing performance requirements, let us define the concept of violation index, as follows. A *violation index* for a p-instance metric μ_I is a function $\sigma_I : \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}$ that expresses how much a performance value deviates from some predefined performance requirements. In particular, this function assigns positive (resp., negative) scores to all the performance values that do not satisfy (resp., that satisfy) the requirements. The absolute value of the score indicates how much the performance value is far from the border between satisfactory and unsatisfactory performance outcomes.

A *single-instance process performance indicator* (*S-PPI*) for a process P is a pair $\phi = \langle \mu_I, \sigma_I \rangle$, such that $\mu_I : \mathcal{I}_P \rightarrow \mathbb{R}_{\geq 0}$ is a p-instance metric for P and $\sigma_I : \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}$ is a violation index. A p-instance $\iota \in \mathcal{I}_P$ is *positive* w.r.t. ϕ iff $\sigma_I(\mu_I(\iota)) > 0$, and *negative* otherwise. In the former case, we also say that ι is a *violation* or a "violating" p-instance w.r.t. ϕ, whereas in the latter, we call ι a normal or "compliant" p-instance w.r.t. ϕ.

For example, consider the p-instance metric μ_I^{CS} and S-PPI $\phi^{CS\geq 8}$ described in Example 1 for process P_{ex}. Denoting as $\sigma^{CS\geq 8}$ a violation index that returns $\sigma^{CS\geq 8}(x) = 8 - x$ for any real performance value x, we can redefine $\phi^{CS\geq 8}$ as follows: $\phi^{CS\geq 8} = \langle \mu_I^{CS}, \sigma^{CS\geq 8} \rangle$. Clearly, any p-instance (e.g., $\iota_1, \iota_2, \iota_4, \iota_6, \iota_7$, or ι_9) having a CS level lower than 8 is positive w.r.t. $\phi^{CS\geq 8}$.

Aggregate metrics and aggregate PPIs. Let ϕ be an S-PPI for a process P. Then, an *aggregate metric* over ϕ is a function of the form $\mu_A : 2^{\mathcal{I}_P} \rightarrow \mathbb{R}_{\geq 0}$ that maps any set X of p-instances of P to a single numeric performance value, by combining the evaluations that the elements of X receive from ϕ, in terms of performance and/or violation scores. To combine the individual contributions of the p-instances in X, some aggregation operator needs to be applied to either X or a subset of X (chosen according some filtering criterion).

For example the aggregate metric of Example 1 counts how many p-instances in a given set are violating (w.r.t. the S-PPI $\phi^{CS\geq 8}$). For the sake of simplicity, let us focus on the case where μ_A is *additive* (i.e. for every set X of p-instances and any partitioning of X in subsets $X_1, X_2, \ldots, X_k$, it is $\mu_A(X) = \sum_{k=i}^{k} \mu_A(X_i)$) and that $\mu_A(\emptyset) = 0$. Anyway, notice that it is easy to extend the framework to deal with algebraic aggregate metrics, as discussed in [2].

Formally, an *aggregate PPI* (short *A-PPI*) is a pair $\Phi = \langle \mu_A, \sigma_A \rangle$, where μ_A is an aggregate metric and $\sigma_A : \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}$ is an associated violation index. Any p-instances set X is said *positive* (resp., *negative*) w.r.t. $\Phi = \langle \mu_A, \sigma_A \rangle$ iff $\sigma_A(\mu_A(X)) > 0$ (resp., $\sigma_A(\mu_A(X)) \leq 0$). Positive p-instance sets are also named (aggregate) *violations* w.r.t. Φ. Finally, let us regard an A-PPI as a function mapping any p-instance set X to a number in $\{-1, 1\}$ such that $\Phi(X) = 1$ (resp., $\Phi(X) = -1$) iff X is positive (resp., negative) w.r.t. Φ.

For instance, with regard to Example 1, let $\mu_A^{CSViol\#}$ be the aggregate metric that maps any set X of p-instances of P_{ex} to the number of p-instances of X that are positive w.r.t. $\phi^{CS\geq 8} = \langle \mu_I^{CS}, \sigma^{CS\geq 8} \rangle$. Then, we can define $\Phi^{CSViol\#\leq 5} = \langle \mu_A^{CSViol\#}, \sigma_A^{CSViol\#\leq 5} \rangle$, assuming that $\sigma_A^{CSViol\#\leq 5}(x) = x - 5$ for any aggregate performance value x. When applied to the window w_i of Figure 1, $\mu_A^{CSViol\#}$ returns the aggregate value 6, which infringes the constraint specified in $\Phi^{CSViol\#\leq 5}$; thus, w_i is positive w.r.t. $\Phi^{CSViol\#\leq 5}$.

Traces and (trace-based) S-PPI prediction. Let E and $\mathcal{T}$ be the universes of all possible events and traces, respectively, that can be produced by the process P. Any *event* $e \in E$ is a tuple storing a reference to the associated p-instance and a timestamp, denoted by $case(e)$ and $time(e)$, and a vector $prop(e)$ of properties.

Any *trace* τ in $\mathcal{T}$ represents the history of a p-instance, encoded as a sequence of events (i.e. all those concerning the p-instance). As for events, $case(\tau)$ (resp., $prop(\tau)$) denotes the p-instance (resp., the vector of data properties) associated with τ. Moreover, $\tau[i]$ is the i-th event of τ, and $\tau(i] \in \mathcal{T}$ is the *prefix* trace consisting of its first i events of τ, for any $i \in \{1 \,..\, len(\tau)\}$, where $len(\tau)$ is the number of events in τ.

A *log* L (over $\mathcal{T}$) is a subset of $\mathcal{T}$, while the *prefix set* of L, denoted by $Prefs(L)$, is the set of all prefix traces that can be derived from L. Finally, $traces(L)$ (resp., $events(L)$) is the set of all the traces in L (resp., all the events stored in L's traces).

A first ("single-instance") prediction problem amounts to estimating the overall performance value of a p-instance, at any moment in its life, based on its current trace. The target of such predictive analysis hence is the (unknown) function $\lambda : \mathcal{T} \rightarrow \mathbb{R}$ that virtually assigns a performance value $\lambda(\tau) = \mu_I(case(\tau))$ to any (possibly partial) trace τ in $\mathcal{T}$ —where $case(\tau)$ is the entire p-instance associated with τ. The basis for making this forecast is the information that τ provides on the history of the respective p-instance. However, as the contents of log traces may be very detailed, they should be brought to a suitable abstraction level, to prevent the discovery of overfitting predictors. This is usually done by resorting to a *trace abstraction* function that converts each trace τ into a (high-level) propositional representation of the execution steps stored in τ.

Definition 1 (PPM). *Let P be a process, over event (resp. trace) universe E (resp., $\mathcal{T}$). Then, a Performance Prediction Model (PPM) for P is a function $M : \mathcal{T} \rightarrow \mathbb{R}$ (approximating λ all over $\mathcal{T}$) that maps each τ in $\mathcal{T}$ with an estimate $M(\tau)$ for $\lambda(\tau)$, by only looking at (an abstracted representation of) the contents of τ.* □

Discovering such a model is an inductive learning problem where a given log L is used as a training set, and $\lambda(\tau)$ is known for each (sub-)trace $\tau \in Prefs(L)$.

Aggregate time-series and time-series predictors. Before defining the kind of time-series predictors used in our work, let us introduce some preliminary notation. In our setting a *time series* is any infinite sequence $\boldsymbol{G} = \langle g_0, \ldots, g_j, \ldots \rangle$ such that all the elements of the sequence belong to $\mathbb{R}^v$, with $v \in \mathbb{N}$. When $v > 1$, we name the sequence a "multidimensional" time series, otherwise we name it a "unidimensional" time series. For any time series $\boldsymbol{G} = \langle g_0, \ldots, g_j, \ldots \rangle$, let $\boldsymbol{G}[i] = g_i$ be the element of $\boldsymbol{G}$ at position i, and $\boldsymbol{G}(i] = \langle g_0, \ldots, g_i \rangle$ be the prefix of $\boldsymbol{G}$ of length $i + 1$. Let $\boldsymbol{G}[i, j]$ be the (contiguous) subsequence $\langle \boldsymbol{G}[i], \boldsymbol{G}[i + 1], \ldots, \boldsymbol{G}[j] \rangle$ of $\boldsymbol{G}$, for any $i, j \in \mathbb{N}$ such that $i < j$. Finally, let $Prefs(G)$ be the set containing all the prefixes $\boldsymbol{G}(i]$ of $\boldsymbol{G}$, and $k\text{-}grams(\boldsymbol{G})$ be the set of all the subsequences of the form $\boldsymbol{G}[i, i + k - 1]$, where $i, k \in \mathbb{N}$.

Definition 2 (TSPM). *Let $\mathbf{Y}$ and $\mathbf{X}$ be a target time series, and an auxiliary one, respectively. Then, a* time-series prediction model *(short* TSPM*) for $\mathbf{Y}$ w.r.t. $\mathbf{X}$, with* horizon z *and* lag l *is a function $M^{TS} : l\text{-grams}(\mathbf{Y}) \times l\text{-grams}(\mathbf{X}) \rightarrow \mathbb{R}^z$ that can predict the z values of $\mathbf{Y}$ that will follow a given prefix of $\mathbf{Y}$, based on the latest l values of $\mathbf{Y}$ and of $\mathbf{X}$. More precisely, for any prefix $\mathbf{Y}(i] = \langle y_0, y_1, \ldots, y_i \rangle$, $M^{TS}(\mathbf{Y}[i-l+1, i], \mathbf{X}[i-l+1, i])$ is a z-gram representing a prediction for $\mathbf{Y}[i+1, i+z]$. Then, for each $j \in \{1, \ldots, z\}$, $M^{TS}(\mathbf{Y}[i-l+1, i], \mathbf{X}[i-l+1, i])[j]$ is the predicted value of $\mathbf{Y}[i + j]$.* □

To predict the values of $\boldsymbol{Y}$ over the next z slots, a TSPM needs to be provided with the l most recent elements of both $\boldsymbol{Y}$ and $\boldsymbol{X}$. This allows for focusing on recent history of a time series, while abstracting from values produced a long time before.

3 PROBLEM AND SOLUTION APPROACH

Given an A-PPI $\Phi = \langle \mu_A, \sigma_A \rangle$ defined for a given process P (based on some S-PPI ϕ), we want to predict whether any window w_i of P will eventually meet the requirement expressed by σ_A, as long as w_i unfolds. Specifically, let Δ be the temporal width of all process windows that need to be considered in the evaluation of aggregate process performances, and t_0 be an initial observation time. Let us also assume, for the sake of presentation, that the prediction is to be made at a given number n of time points, named *checkpoints*, within the current window, which correspond to fixing an estimation rate $\delta = \frac{\Delta}{n}$. Checkpoints split the process windows into smaller segments, named *slots*. Then, for each current window w_i, we must predict whether w_i will be positive w.r.t. Φ or not.

The stream of p-instances generated by the process under monitoring produces two series of sets: process windows $\langle w_0, \ldots, w_i, \ldots \rangle$, of granularity Δ, and process slots $\langle s_0, \ldots, s_j, \ldots \rangle$, of granularity $\delta = \frac{\Delta}{n}$. As shown in Figure 1, the slots extracted from process window w_i are also numbered as $s_0^i, s_1^i, \ldots, s_{n-1}^i$.

Let t_j^i be the j-th checkpoint within window w_i. Then, for each i in $\mathbb{N}^+$ and each $j \in \{0, \ldots, n-1\}$, we want to estimate if w_i will be positive w.r.t. Φ, by using summary data about P's history

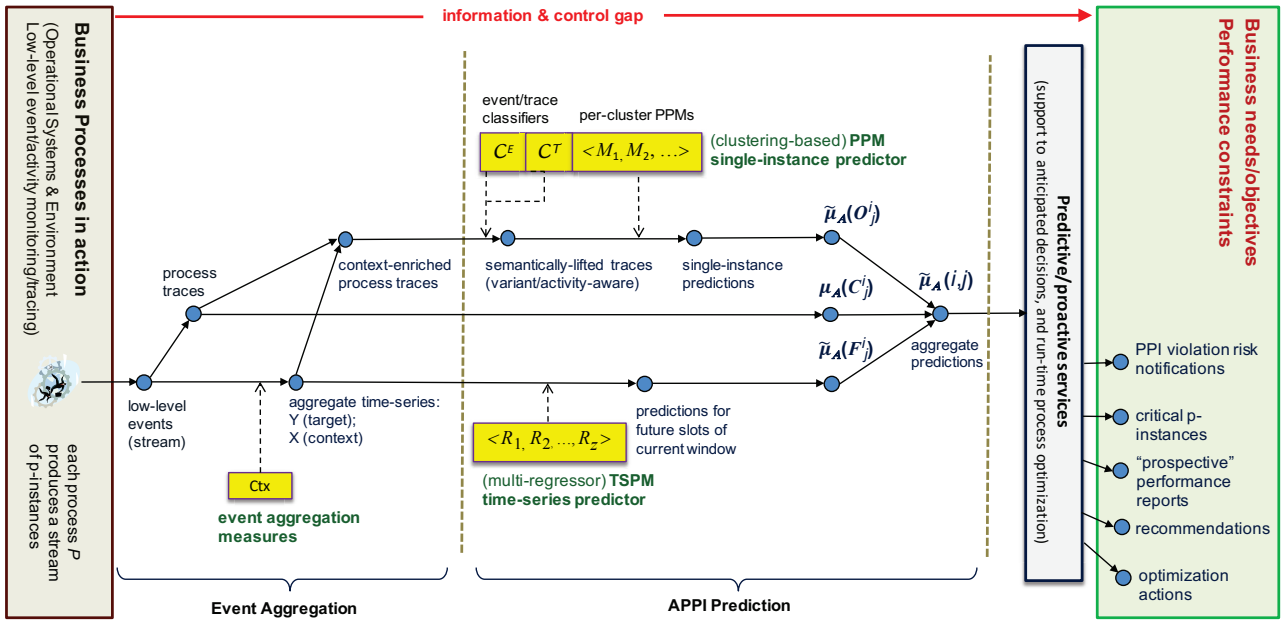


Figure 2: From low-level events/data to the predictive/proactive monitoring of process performances.

and trying to guess how P is evolving in the rest of w_i. The prediction problem at any checkpoint t_j^i is decomposed into three sub-problems, regarding w_i as the union of three disjoint sets: *(i)* the set C_j^i of w_i's p-instances finished before t_j^i; *(ii)* the set O_j^i of w_i's p-instances that started before t_j^i but have not finished yet; and *(iii)* the set F_j^i of w_i's p-instances that will start after t_j^i. These sets are shown in Figure 1, for window w_i and checkpoint $t_2^i = t_{k+2}$.

Let $\tilde{\mu}_A(i,j)$ be the forecast that is to be made at checkpoint t_j^i, for the aggregate measurement $\mu_A(w_i)$. Then, under the working assumption μ_A is additive, we can compute $\tilde{\mu}_A(i,j)$ as follows:

$$\tilde{\mu}_A(i,j) = \mu_A(C_j^i) + \tilde{\mu}_A(O_j^i) + \tilde{\mu}_A(F_j^i) \quad (1)$$

where the $\mu_A(C_j^i)$ is the result taken by μ_A on the "completed" p-instances in C_j^i, while $\tilde{\mu}_A(O_j^i)$ and $\tilde{\mu}_A(F_j^i)$ are the aggregate value of μ_A estimated for the sets of "ongoing" and "future" p-instances, respectively. As proposed in [9], predictions for $\tilde{\mu}_A(O_j^i)$ and $\tilde{\mu}_A(F_j^i)$ can be obtained by resorting to PPM and a TSPM, of the forms defined in the previous section.

To support the computation of $\tilde{\mu}_A(i,j)$, we exploit an inductive learning framework (depicted in Figure 2 and extending the one proposed in [2, 9]) that allows us to estimate the latter two terms in Equation 1, based on historical event data.

Basically, Figure 2 sketches a two-stage (with the stages separated by a vertical dashed line) cascade conceptual scheme for the predictive monitoring of an aggregate PPI: raw process events are first processed (left side of Figure 2) in order to bring them at a higher level of aggregation, and the resulting aggregate data are then used to compute $\tilde{\mu}_A(i,j)$ (right side of Figure 2).

Notice first that the estimation of $\tilde{\mu}_A(O_j^i)$ is accomplished by applying a clustering-based PPM predictor (learnt from historical traces) to each of the (partial) traces that have been gathered for the ongoing p-instances in O_j^i. Clearly, the same approach cannot be exploited for predicting the performances of the future p-instances of F_j^i, since we have no real data on their actual structure and characteristics. However, we can discover a "black-box" predictive model from the series of aggregate values that μ_A has generated up to the current checkpoint, for all the past slots.

Let us denote hereinafter this *target* time series as $\boldsymbol{Y}$, while assuming that another (possibly multidimensional) *auxiliary* time series $\boldsymbol{X}$ can be computed that stores context-oriented aggregate data for each slot, and exploited to forecast future values of $\boldsymbol{Y}$ (i.e. values that μ_A will take on future slots). As shown in Figure 2, the discovered time-series predictor is exploited to predict the value of μ_A over all the future slots of the current window. These per-slot estimates are eventually combined together to obtain $\tilde{\mu}_A(F_j^i)$.

Based on these numerical forecasts, at checkpoint t_j^i window w_i is predicted to be positive w.r.t. $\Phi = \langle \mu_A, \sigma_A \rangle$ iff $\sigma_A(\tilde{\mu}_A(i,j)) > 0$.

Both the PPM and TSPM models need to be properly fed with suitable process traces and historical values of the time series (namely, $\boldsymbol{Y}$ and $\boldsymbol{X}$), respectively. To this end, as shown in the left side of Figure 2, low-level process events are first grouped into (possibly partial) traces. These events are also exploited to associate their respective slots with summary data, computed through suitable "event aggregation measures", so materialising an auxiliary context-oriented time series $\boldsymbol{X}$. The actual performances of finished traces are also used to update the values (one per slot) of the target time series $\boldsymbol{Y}$. The tuple $prop(\tau)$ of each process trace τ is also enriched with context data extracted from $\boldsymbol{X}$—precisely, with the summarised values associated with the slot to which τ belongs.

The special kinds of PPM and TSPM adopted in our approach. In order to deal with logs of lowly-structured processes where each trace

just stores a sequence of fine grain generic operations, in the current implementation of our framework, we leveraged the PPM learning approach proposed in [8], This allows us to discover a clustering-based PPM, based on two different classification functions, one (denoted by C_E) allowing for abstracting raw log events into relevant event classes, and the other (denoted by C_T) for discriminating among different trace classes (playing as different execution scenarios). In such an articulated PPM model, each trace class is equipped with a specialised "cluster-wise" PPM, obtained by applying any standard regression algorithm to a bag-oriented propositional representation of the traces, where any event sequence is projected onto the space of the discovered event classes.

Regression-based induction algorithms are also exploited to learn the TSPM component, which, as illustrated in the figure, takes the form of a battery $< R_1, \ldots, R_z >$ of simpler "single-target" TSPMs (discovered each by using a standard regression method), such that element R_k (for $k \in [1..z]$) is specialised to make a direct forecasts for $\boldsymbol{Y}$, k steps ahead in the future.

Further details on such PPM and TSPM models (and on they can be discovered from a given log) can be found in [2].

Event aggregation measures and resulting aggregate time-series $\mathbf{Y}$ *and* $\mathbf{X}$. Before leaving this section, let us give some details on the slot-based aggregation measures that can be used to compute the auxiliary time series $\boldsymbol{X}$. The events contained in a slot s_k provide factual evidence on how the business processes and their surrounding environment behave in the time interval $[start(s_k), end(s_k))$. Such events can be turned into summary context-oriented variables by defining ad-hoc aggregation-based event transformation functions, which play as the event-data counterpart of the aggregate (performance) metrics that we defined for sets of p-instances.

An *event aggregation measure*, w.r.t. event universe E, is a function of the form $\omega : 2^E \rightarrow \mathbb{R}_{\geq 0}$ that maps any subset s of E to a numeric value $\omega(s)$, based on the contents of the events in s. The value can be computed, in general, by taking account for the information stored in each of the events of s.

Let us focus in the following on *additive* event measures ω such that $\omega(\emptyset) = 0$ expressing process-oriented workload measures, such as: the total number of p-instances started in s, and the number of events of s that refer to a given process/organisation entity. In general, one can use several event aggregation measures, say $\omega^1, \ldots, \omega^v$ to associate an event slot with multiple summary statistics. Thus, extending the definition above, a "multidimensional" event aggregation measure $Ctx(\cdot) = \langle \omega^1(\cdot), \ldots, \omega^v(\cdot) \rangle$ can be defined to map each slot s to the tuple $Ctx(s) = \langle \omega^1(s), \ldots, \omega^v(s) \rangle$ of numerical aggregate values.

In fact, it is the iterated application of Ctx to slots $\langle s_0, s_1, \ldots \rangle$ produced by the process under monitoring that yields an auxiliary time series $\boldsymbol{X} = \langle x_0, x_1, \ldots \rangle$, which is exploited in our approach to better predict future values of the target one $\boldsymbol{Y} = \langle y_0, y_1, \ldots \rangle$ —where y_i is the value taken on slot s_i by some chosen aggregate metric μ_A for $i \in \mathbb{N}$.

4 ONLINE PREDICTIVE MONITORING AND LEARNING ALGORITHM

A-PPI prediction model (APPM). Before illustrating the algorithm, let us formally define the kind of integrated prediction model that it is based upon. The model will be eventually used to predict whether the current process window, say w_i, will be positive w.r.t. Φ or not, at whatever observation time along the unfolding of w_i. To let the reader benefit from the schema in Figure 1, let us focus on the case where the observation time coincides with one of the predefined checkpoints of w_i, say t^i_j.

Definition 3 (*APPM).* Let $\Phi = \langle \mu_A, \sigma_A \rangle$ be an A-PPI for a process P. Let $\Delta, t_0 \in \mathbb{R}_{\geq 0}$ and $n \in \mathbb{N}$ be the chosen window duration, initial time and number of checkpoints per window. Let w_i be the current process window and t^i_j be the current prediction time (coinciding with one of the checkpoints of w_i). Then, an *Aggregate Performance Prediction Model* (*APPM*) for Φ and t^i_j (w.r.t. Δ, t_0 and n, omitted for short) is a tuple of the form $H = \langle M^P, M^{TS}, Ctx, oTraces, X, Y, val^C, val^O, val^F \rangle$, such that:

- M^P and M^{TS} are an PPM for P and an TSPM for $\boldsymbol{Y}$ w.r.t. $\boldsymbol{X}$, respectively;
- Ctx is a vector of event aggregation measures (for computing per-slot context data);
- $oTraces$ is a data index storing one triple $\langle c, \tau, p \rangle$ for each p-instance ι of w_i that is ongoing at current time t^i_j, such that: c is the identifier of ι, τ is the partial trace available for ι, and p is an updated prediction for the $\mu(\iota)$ (obtained by applying M^P to τ); $oTraces$ provides three access methods: *(i)* $oTraces.search(c)$, which returns a pair $\langle \tau, p \rangle$ if the index contains a triple $\langle c, \tau, p \rangle$, or a "dummy" pair $\langle \langle \rangle, 0 \rangle$ otherwise; *(ii)* $oTraces.insert(c, \tau, p)$, for inserting a new triple (c, τ, p) into the index; and *(ii)* $oTraces.remove(c)$, for removing the triple associated with c, if any, from the index;
- X, Y are two lists of real numbers storing a (partial) materialisation of the time series $\boldsymbol{X}$ and $\boldsymbol{Y}$ up to checkpoint t^i_j;
- $val^C \in \mathbb{R}_{\geq 0}$ is a variable storing the actual value of the aggregate metric μ_A on the completed p-instances of the current window w_i and prediction time t^i_j (i.e. $val^C = \mu_A(C^i_j)$);
- $val^O, val^F \in \mathbb{R}_{\geq 0}$ are two variables storing the aggregate predictions for the ongoing and future p-instances of w_i (i.e. $\mu_A(O^i_j)$ and $\mu_A(F^i_j)$), respectively;

Model H is associated with a function, denoted as $H.pred()$, which provides a prediction (updated to checkpoint t^i_j) for the event that the current window w^i will be positive w.r.t. Φ or not, computed as: $H.pred() = 1$ iff $\sigma_A(val^C + val^O + val^F) > 0$, and $H.pred() = -1$ otherwise. Hence, the value returned is 1 (resp., -1) when w_i is estimated to be positive (resp., negative) w.r.t. Φ. □

The components M^P, M^{TS} of an *APPM* are the two mutually complementary predictors for dealing with the ongoing and future p-instances of a window, respectively. By contrast, components $oTraces, X, Y, val^C, val^O, val^F$, named hereinafter as the *state variables* of H, provide summary information on both the event data collected till the current checkpoint t^i_j and fresh predictions for the ongoing and future p-instances of the current window w.r.t. the same checkpoint t^i_j. The overall prediction function $H.pred()$ depends on the internal state of the APPM H (in particular, on val^C, val^O and val^F), which is assumed to reflect the data and predictions available at the current checkpoint t^i_j; indeed, $H.pred()$ is

meant to be called continuously, as soon as a new event becomes available, as a sort of continuous (predictive) query.

Online Learning and Prediction Algorithm. Figure 3 illustrates our approach to the predictive monitoring of a given A-PPI $\Phi = \langle \mu_A, \sigma_A \rangle$, based on streaming events, under the working assumption that μ_A combines the results of an S-PPI $\phi = \langle \mu, \sigma \rangle$ through an additive aggregation function. The approach is shown as an algorithm, named `APPM-EventProc`, which is meant to be run on a continuous per-event fashion. The main input to the algorithm is indeed an event e that can represent either any of the event records triggered by the execution of an activity of the process, say P, under monitoring (and captured by suitable tracing mechanisms), or a special "artificial" *end-slot* event, generated by an ad-hoc timer, that marks the temporal end of any new slot resulting from segmenting the stream of P's events (according to the window duration Δ, initial time t_0, and number n of slots per windows).

The information stored in e is exploited to eventually return an updated prediction for the current process window, denoted as w_i, and the lastly-elapsed checkpoint of its, denoted t^i_j, estimating whether w_i will eventually be violating/positive w.r.t. Φ or not (the prediction returned is 1 in the former case, and -1 in the latter). The prediction is made with the help of an ensemble of APPMs, represented here as a list $\mathcal{H}$, which is assumed to be initialised as to contain just an existing APPM h_1. In the tests discussed in Section 5, model h_1 was always induced from historical event data, by using a variant (discussed in more detail later on) of the offline discovery algorithm defined in [2, 9].

The same offline learning scheme `APPM-mine` is exploited to implement the procedure `extendEnsemble`, which allows the algorithm to expand the current ensemble $\mathcal{H}$ with a new APPM, every time a new batch ΔL of training data becomes available, such that ΔL covers a fixed number *batchSize* of contiguous slots over the stream of events that have been processed so far by the algorithm. In fact, parameter *batchSize* (assumed to be a multiple of n) indicates how many slots are needed to learn any novel APPM of $\mathcal{H}$, and hence it allows the algorithm to control the frequency at which $\mathcal{H}$ is updated. Essentially, the update of $\mathcal{H}$ follows a batch-based incremental learning scheme, where the continuous stream of events generated by process P is segmented into "consolidated" sub-logs $\Delta L^{(1)}, \Delta L^{(2)}, \ldots$, such that $\Delta L^{(1)}$ covers the first block of *batchSize* contiguous slots that have been processed by the algorithm, $\Delta L^{(2)}$ is the subsequent of such blocks, and so on. In order to build these sub-logs, named *batches* hereinafter, all the events processed by the algorithm are kept in a "buffer" log L, which constitutes a "global" variable persisting over different calls to the algorithm. As soon as L is reckoned to contain a new batch, the latter is saved into a local variable ΔL, and passed to the auxiliary procedure `extendEnsemble`, in order to induce a new element of the ensemble $\mathcal{H}$ (steps 3-7). We pinpoint that the call to `extendEnsemble` does not block the execution of algorithm `APPM-EventProc`, since each batch learning step is implemented as a separate processing thread that runs concurrently with `APPM-EventProc`.

Three further global variables are used by `APPM-EventProc` to update and use the APPM ensemble $\mathcal{H} = \langle h_1, \ldots, h_T \rangle$ (where T is the number of batches processed up to the current event): *(i)* two lists $S\#$ and $E\#$ storing how many p-instances were opened and closed for each past slot (precisely, $S\#[i]$ and $E\#[i]$ are the of numbers opened and closed p-instances for any slot s_i, respectively); *(ii)* a list $A = \langle a_1, \ldots, a_T \rangle$ where each a_k provides sufficient statistics for computing a time-adjusted loss for APPM h_k, over all the per-checkpoint predictions that have been made with h_k. The lists $S\#$ and $E\#$ allow the algorithm to check whether a new batch of fresh training data has become available in the buffer log L (cf. step 3), and need to be materialised only over the slots that have not been used yet to build a batch of training data for learning an APPM. The loss scores in A (updated incrementally with the help of procedure `extendEnsemble`) are employed to associate each model h_k of the ensemble with a dynamical accuracy-oriented weight w_k, and eventually return an updated overall prediction *ensemblePred* for the current window w_i as a weighted combination of the predictions made for w_i by all the APPMs in $\mathcal{H}$ (steps 35-39).

Let us now summarise the algorithm's steps that have not discussed so far. Basically, steps 9-13 are devoted to update the state of each APPM in $\mathcal{H}$ (in particular, the time-series components X, Y and partial aggregate performance scores val^C, val^O and val^F), as to take account for the fact that a new slot s^i_j has just elapsed –indeed, these steps are only performed when the input event e just represents an end-slot marker (i.e. when $j = n$).

By contrast, steps 14-34 describe the processing logics for the case when e is a real business processing event that conveys information on the execution of a step for some instance, say ι, of the process under monitoring. At start, the algorithm must decide whether e is the first event generated by ι or not. This task is accomplished by simply checking (at step 16) whether the *oTraces* index of any of the APPMs of $\mathcal{H}$ contains a trace, say τ, associated with the same case identifier as e (and storing the previous events of ι); this allows indeed the algorithm to spot the slot, say s_m, to which ι belongs, based on either $time(\tau[0])$ (if a non empty trace τ is in the index), or $time(e)$ (if e is the first event of ι). In case e is a starting (resp., final) event of ι, the counter $S\#[m]$ (resp., $E\#[m]$) is incremented, relatively to slot s_m (steps 18-19).

Steps 20-33 are meant to update the state of each APPM h_k in the ensemble $\mathcal{H}$, relatively to both its *oTraces* index, and the variables var^O and var^C (in case e is a final event). To this end, a search (step 21) is performed in the index *oTraces* to retrieve the trace τ currently stored for ι (if any, otherwise the returned trace τ will be an empty trace), and the most recent prediction p that was made for ι by h_k, before processing e. In any case, a new trace τ' is obtained for ι by appending e to the event list of τ, while storing a new estimate for the performance outcome of ι into v' —in particular, when e is the last event of ι, v' will store the ultimate performance value $\mu(\iota)$ of ι. In particular, step 32 updates, relatively to slot s_m, both the target time series Y (based on the novel estimate of $\mu(\iota)$ stored in v') and the auxiliary one X (based on the result of applying the aggregation measure *Ctx* to e).

The last five steps compute and eventually return a fresh prediction for the current window w_i, by combining the predictions of the APPMs in $\mathcal{H}$ via the weighting scheme discussed before.

Procedure `extendEnsemble`. This procedure (not reported here in detail for lack of space and running as a concurrent subprocess of the main algorithm `APPM-EventProc`) is meant to enrich a given ensemble $\mathcal{H}$ of APPMs with the addition of a new APPM, and to

Input: Event $e \in E$; *// e can be either a process event or an artificial "timer-generated" end-slot event*

Parameters: natural numbers $batchSize$ (multiple of n) and $maxSize$ indicating how many slots to use to learn any APPM and the maximal length of $\mathcal{H}$, respectively; window duration Δ, initial time t_0, number n of slots per windows; A-PPI $\Phi = \langle \mu_A, \sigma_A \rangle$ such that μ_A is defined in terms of a given p-instance metric μ_I; vector Ctx of event aggregation measures; base learning methods $basePPM$ and $baseTSPM$; natural numbers z, l indicating the desired TSPMs' horizon and lag, respectively;

Global variables: "buffer" log L containing previous events not used yet to learn an APPM; list $\mathcal{H}$ of *APPM*s such that $h_k = \mathcal{H}[k]$ is the model induced from the k-th data batch, for $k \geq 2$; list A such that $a_k = A[k]$ stores time-adjusted loss statistics (cf. [5]) for h_k over the predictions made with it; lists $S\#$ and $E\#$ storing how many p-instances were opened and closed, respectively, for each slot;

Output: updated prediction for the current window at the arrival of e.

Steps:

1 **if** e is an end-slot event **then**
2 let $s_{i'}, s_{i'+1}, \ldots, s_{i'+q}$ be the (contiguous) sequence of slots covered by L; *// for each $j \in [i'..i'+q]$, slot s_j consists of all the p-instances of L that started in the time range $[t_0 + j \times \Delta/n,\ t_0 + (j+1) \times \Delta/n]$;*
3 **if** $S\#[j] = E\#[j]\ \forall j \in [i'..i' + batchSize - 1]$ **then**
4 let ΔL be the subset of events in L associated with p-instances of the slots $s_{i'}, \ldots, s_{i'+batchSize-1}$;
5 `extendEnsemble`(ΔL, $\mathcal{H}$, A, $basePPM$, $baseTSPM$, Φ, Δ, t_0, n, z, l, $maxSize$); *// non-blocking call*
6 $L := L - \Delta L$;
7 **end if**
8 let i be the index of the process window currently under monitoring and $s^i_j \equiv s_{j'}$ be the slot just elapsed;
9 **for each** APPM h_k in $\mathcal{H}$ **do**
10 append a new element of value 0 to both $h_k.Y$ and $h_k.X$;
11 $h_k.val^F = \sum_{q=j'+1}^{n} h_k.M^{TS}(Y[j'-l+1, j'], X[j'-l+1, j'])\,[q-j']$;
12 **if** $j = n$ **then** $h_k.val^C := 0$; $h_k.val^O := 0$; **end if**
13 **end for**
14 **else** *// e is a real business processing event*
15 let ι be the p-instance originating e and s_m be the slot containing ι;
16 $isFinal :=$ `true` iff e marks the end of ι;
17 $isStart =$ `false` iff ι is referred to by one of the traces stored in the $oTraces$ index of any APPM in $\mathcal{H}$;
18 **if** $isStart$ **then** $S\#[m] := S\#[m] + 1$;
19 **if** $isFinal$ **then** $E\#[m] := E\#[m] + 1$;
20 **for each** APPM h_k in $\mathcal{H}$ **do**
21 $\langle \tau, v \rangle := h_k.oTraces.search(case(e))$;
22 $\tau' = \tau \oplus e$; *// $\oplus$ stands for sequence concatenation*
23 **if** $isFinal$ **then**
24 set v' to the actual performance value $\mu_I(\iota)$ (computed by looking at the fully unfolded trace of ι);
25 $h_k.val^C := h_k.val^C + \mu_A(\{v'\})$; $h_k.val^O := h_k.val^O - \mu_A(\{v\})$;
26 $h_k.oTraces.remove(case(e))$;
27 **else**
28 $v' := h_k.M^P.predict(\tau')$;
29 $h_k.val^O := h_k.val^O - \mu_A(\{v\}) + \mu_A(\{v'\})$;
30 $h_k.oTraces.insert(\langle case(e), \tau', v' \rangle)$;
31 **end if**
32 $h_k.Y[m] := h_k.Y[m] - \mu_A(\{v\}) + \mu_A(\{v'\})$;
$h_k.X[m] := h_k.X[m] + Ctx(\{e\})$;
33 **end for**
34 **end if**
35 let $\mathcal{H} = \langle h_1, \ldots, h_T \rangle$ and $A = \langle a_1, \ldots, a_T \rangle$;
36 $w_k := \log(1/a_k)$ for $k \in [1..T]$; *// compute the ensemble weights as in [5])*
37 $ensemblePred := \mathrm{sgn}\left(\sum_k^T w_k \cdot h_k.pred()\right)$;
38 **if** $ensemblePred = 0$ **then** $ensemblePred := -1$;
39 **return** $ensemblePred$;

Figure 3: Algorithm `APPM-EventProc`. $\mathcal{H}$ is assumed to initially contain one APPM. For the sake of compactness, μ_A is regarded here as a function over p-instances' performance results (rather than as a function over p-instances), and $case(e)$ is used to denote the ID of the p-instance associated with e (rather than a reference to the p-instance itself).

modify the associated list A accordingly (i.e. to make A contain the right loss statistics for all the elements in the updated version of the ensemble). The discovery of the new APPM and the update of the APPMs' loss statistics rely on using a log ΔL, which plays

as a novel block of training data, providing complete information on *batchSize* contiguous slots and on their composing p-instances (including their actual performance outcomes). The scheme used by `APPM-EventProc`, similar to that of the online classification algorithm $Learn^{++}.NSE$ [5], relies on exploiting a variant of the offline learning method proposed in [2] to induce an APPM from ΔL.

Basically, the APPM induction method consists of three phases [2]:

(1) Two optimal classification functions C_E and $C_{\mathcal{T}}$ are induced synergistically for the events and traces of ΔL, respectively, through a "co-clustering" procedure where a predictive clustering function is iteratively applied to the propositional encodings of ΔL's events and, alternately, of ΔL's traces — including, for each event/trace, the context data (computed with *Ctx*) of its respective slot.
(2) A clustering-based PPM is built, by equipping each of the discovered trace clusters with a distinct predictor (i.e. a PPM), induced by applying the *basePPM* method to a propositional encoding of the traces in the cluster including both slot-oriented context data and a bag-oriented representation of their event sequences (after abstracting each event into an event class with the help of C_E).
(3) A multi-regressor TSPM is built for the target series $\boldsymbol{Y}$ (w.r.t. $\boldsymbol{X}$) by applying the learning method *baseTSPM* to z different propositional views extracted from both X and Y —precisely, for each past step i of both time series, a distinguished training tuple is put in the k-th view (for $k \in [1..z]$) that stores the subsequences $Y[i-l, i-1]$ and $X[i-l, i-1]$ as predictor variables, and $X[i+k-1]$ as the only target variable to be predicted. As a result a battery of z single-target regression models, say $R_1, \ldots, R_z$, is built, where each R_k can make a direct forecasts for $\boldsymbol{Y}$, k steps ahead in the future.

Beside putting the newly discovered APPM in $\mathcal{H} = \langle h_1, \ldots, h_T \rangle$, procedure `extendEnsemble` removes the oldest model h_1 from $\mathcal{H}$ and the corresponding element a_1 from A, in case $\mathcal{H}$ has already reached the maximum size (i.e. if $T = maxSize$). Moreover, it exploits the approach of [5] to initialise/update the loss statistics of all the APPMs (regarded as classifiers) in $\mathcal{H}$, based on their classification errors over ΔL. These statistics (eventually used to derive the voting weights of the APPM classifiers in $\mathcal{H}$) are computed using a "time-adjusted" loss function [5] that returns a sort of sigmoidal average over the classifiers' losses and favours those that have performed well recently. Specifically, first a "sampling" weight is associated with each checkpoint (representing a distinguished training instance) in ΔL, based on the classification errors that $\mathcal{H}$'s APPMs make for them (cf. [5]). Then, for each h_k in $\mathcal{H}$ a weighted overall classification error ϵ_k^T over the checkpoints in ΔL is computed (taking into account their associated weights), and the list A is updated on the basis of the errors $\epsilon_1^T, \ldots, \epsilon_T^T$ (precisely, each a_k is assigned the value of the coefficient $\overline{\beta}_k^T$ defined in [5]).

5 EXPERIMENTS

In all the tests, we fixed the duration of the process windows to $\Delta = 7$ days and $n = 7$ checkpoints per window. This resulted in partitioning the log into a sequence of n_T slots $[s_0, \ldots, s_{n_T-1}]$, of duration $\delta = \Delta/n = 1$ day each. We then grouped the slots into sub-logs $[S_{L1}, \ldots, S_{Lp}]$ (with $1 \leq i \leq p$), named "batches", such that each batch gathered $\lfloor (batchSize\% * n_T)/100 \rfloor$ contiguous slots. In the experimentation, we specifically made *batchSize%* to vary in {5%, 10%, 15%, 20%} of n_T.

In each test (once fixed *batchSize%*), we used the contents of S_{L1} to induce an initial APPM h_1, and the remaining batches $S_{L2}, \ldots, S_{Lp}$, regarded as a collection L_{TS} of streaming events, to test our approach, iteratively running `APPM-EventProc` on all the events of L_{TS}, ordered temporally. According to the scheme in Figure 3, the ensemble $\mathcal{H}$ (initially containing only h_1) is extended dynamically by adding a new APPM as soon as a new batch of L_{TS} appears to be complete. As to `APPM-EventProc`'s parameters, we fixed $maxSize = 100$, and instantiated the learning methods *basePPM* and *baseTSPM* (for learning clusters' PPMs and time-series regressors, respectively) with an incremental implementation of classic *Linear Regression* method (denoted hereinafter as *LR*), based on the *stochastic gradient descent (SGD)*, for the sake of efficiency. As to the *baseTSPM*'s parameters, we set $l = 10$ and $z = n + \lceil T_{90p}/\delta \rceil$, where T_{90p} is the 90th percentile of the durations of L's traces.

As terms of comparison for `APPM-EventProc`, we considered two baseline approaches, namely `Baseline_1`, and `Baseline_2`. `Baseline_2` just corresponds to applying the offline learning scheme proposed in [2] (using *LR* as base regression method) to the first batch S_{L1}, hence considered as the only training data available; the *APPM* discovered this way, was then tested against L_{TS} (containing the remaining n_{TS} slots).

`Baseline_1` simulates a more powerful variant of the approach in [2] featuring online-learning capability. Specifically, exploiting the fact that the base regression method *LR* is incremental, the discovered *APPM* is allowed to update all of its PPM sub-models on a "per-instance" basis: as soon as a p-instance ι ends, all of its prefix traces (labeled with ι's performance value) are used to update the PPM of the cluster ι was assigned to.

Evaluation Measures. Let $\mathcal{W}_{TS}$ be the sequence of m windows (each of duration Δ) extracted from the testing log L_{TS}, with $m = n_{TS}/n$. For the sake of comparison, we tested all the approaches (i.e. the one proposed and the two baselines) on L_{TS}, focusing only on the predictions that they made correspondingly to each of the n_{TS} checkpoints covered by L_{TS}, say t_j^i, to estimate whether the window w_i in $\mathcal{W}_{TS}$ was violating or not. As algorithm `APPM-EventProc` returns such a prediction for each event e of L_{TS}, for any window w_i in $\mathcal{W}_{TS}$ and any checkpoint t_j^i (such that $j \in [1..n]$), we only considered the last prediction returned before t_j^i.

To evaluate the accuracy of each approach, we regarded violation prediction as a binary classification problem, and measured the ability of the models to discriminate negative vs. positive windows in $\mathcal{W}_{TS}$. Specifically, for each window w_i in $\mathcal{W}_{TS}$ and each checkpoint t_j^i, we assessed whether the predictions made for w_i at t_j^i were correct or not, and computed the classical *precision* ($\mathcal{P}$), *recall* ($\mathcal{R}$), and *f-measure* ($\mathcal{F}$) measures. In addition, as the log may well be imbalanced (since violating windows are usually far less than normal ones), we also computed the *g-mean* ($\mathcal{G}$) measure [12].

Dataset and target A-PPIs. To assess the validity of our prediction approach, we conducted a series of tests on the log of a real-life problem management system used in Volvo IT Belgium, which consists of 7,841 events, pertaining 1,271 process traces, gathered

from January 9, 2011 to May 31, 2012. According to the procedure described before, we segmented the log into $n_T = 508$ slots. Each event in this log owns 8 attributes (namely, `status`, `substatus`, `resource`, `res_country`, `support_team`, `functional_division`, `org_line` and `org_country`), while the trace of each problem p stores two attributes: p's `impact` and the `product` affected.

Let us now describe how we set the target A-PPI $\Phi = \langle \mu_A, \sigma_A \rangle$ and its underlying S-PPI $\phi = \langle \mu_I, \sigma_I \rangle$ in the application scenario. The single-instance metrics μ_I was defined as the *total time* needed to completely process a case, and μ_A as the (additive) function counting the number of p-instances that are deemed to be positive (as explained in the following) w.r.t. the S-PPI.

Since μ_I (resp., μ_A) represents a cost measure, we defined the violation index (cf. Sec. 2) σ_I (resp., σ_A) as the difference between the value returned by μ_I (resp., μ_A) and an upper threshold θ_I (resp., θ_A). Different values were chosen for θ_I and θ_A, in order to consider different amounts of violations at the single-instance or aggregate level, respectively (i.e. different amounts of positive p-instances and windows, respectively). Specifically, we considered three target violation percentages, namely {10%, 20%, 30%}, and for each percentage VP, we set θ_A (resp. θ_I) to the percentile of order VP over all the windows (resp. p-instances) in the log.

5.1 Test results

Accuracy results. Looking at the results shown in Table 1, it is evident that our approach was able to make accurate predictions for the windows. Indeed, algorithm `APPM-EventProc` generally obtained good achievements (whatever the setting) in terms of *recall*, *f-measure* and *g-mean*, sometimes at the cost of some slightly lower *precision*. In particular, the achievements obtained when considering the highest percentage of violation (i.e. $VP = 30\%$) are remarkable, especially as far as concerns the recall score (0.938). In fact, even though good levels of precision are welcome, ensuring a high level of recall is of utmost importance in a violation detection setting (e.g., to undertake timely countermeasures), while high precision is useless when many violating windows are missed.

As to the comparison with the baselines, it is easily seen that `APPM-EventProc` always overcomes both baselines over all the evaluation measures, independently of the chosen percentage VP of violations. In particular, it leads with a pretty wide margin over `Baseline_2`, using a static learning scheme, hence confirming the importance of adopting a continuous learning strategy. On the other hand, the purely incremental learning scheme of `Baseline_1` looks less effective than our ensemble-based approach.

This behaviour becomes even more interesting when considering that the values in Table 1 were computed by averaging the results obtained with different values of $batchSize\%$. Indeed, the robustness of `APPM-EventProc` even with lower values of $batchSize\%$ (namely, $batchSize\% = \{5\%, 10\%\}$) unveils its ability to compensate the lack of an adequate amount of historical data, for effectively initialising the A-PPI prediction model, with its capability to update the model dynamically.

Finally, Table 1 allows us study how the behaviours of the approaches change when varying the percentage VP of violations. Clearly, all the accuracy scores improve when increasing VP, and they reach their maximum with $VP = 30\%$. Such a behaviour is not surprising, as, intuitively, the more violations in the log, the easier they are to reckon. Notably, algorithm `APPM-EventProc` seems to be far more robust than the baselines with respect to the reduction of VP, and it achieves appreciable results event in the critical setting when only 10% of the windows/p-instances are positive. This observation is confirmed by the high g-mean scores achieved by our approach. Indeed, g-mean is commonly used to assess a classifier's accuracy when the classes are imbalanced, as it is in our setting when $VP = 10\%$, and the other accuracy measures might return a distorted overestimated picture of the approaches' performances (mainly influenced by the predictions made over the normal windows, which collectively form the majority class).

VP	Method	$\mathcal{P}$	$\mathcal{R}$	$\mathcal{F}$	$\mathcal{G}$
0.1	`Ours`	**0.661**	**0.850**	**0.742**	**0.898**
	`Baseline_1`	0.308	0.662	0.420	0.744
	`Baseline_2`	0.315	0.503	0.384	0.664
0.2	`Ours`	**0.686**	**0.844**	**0.756**	**0.881**
	`Baseline_1`	0.433	0.685	0.530	0.747
	`Baseline_2`	0.469	0.561	0.498	0.689
0.3	`Ours`	**0.827**	**0.938**	**0.879**	**0.922**
	`Baseline_1`	0.639	0.748	0.689	0.774
	`Baseline_2`	0.748	0.584	0.650	0.724

Table 1: *Precision*, *recall*, *f-measure*, **and** *g-mean* **scores obtained by algorithm `APPM-EventProc` (here renamed `Ours`) and by the baselines for different values of the violation percentage VP. Each value was computed by averaging those obtained with different values of** $batchSize\%$ **(namely,** 5%, 10%, 15%, 20%**). The best achievements are shown in bold.**

Efficiency results. In the online monitoring environment considered in this work, a major requirement is that the average times needed to process an event (with algorithm `APPM-EventProc`) and a batch of training data (with procedure `extendEnsemble`) are low enough compared to the rates at which the events and batches, respectively, are produced. All the tests were performed on a dedicated computer, equipped with an Intel Core i5 at 2.6 GHz and 16 GB (DDR3 1600 MHz) of RAM, and running Mac OS 10.10.5.

In order to assess the efficiency of our approach in stressing conditions, we artificially increased the log size by replicating each log event e Θ times (with Θ in {1, 2, 4, 8, 16, 32, 64}), while perturbing the timestamp of e's copies randomly (of 12 hours at most). Notably, this resulted in multiplying, by a factor Θ, the original average event rate of the log (namely, 15.43 events per slot). In all the tests we fixed $batchSize\% = 20\%$ (representing the heaviest load for procedure `extendEnsemble`) and set $VP = 20\%$ —actually, the setting of VP does not impact on computation times.

For each value of the multiplying factor Θ, Figure 4 reports the average times that were taken, in the test scenario, by algorithm `APPM-EventProc` (to process an event) and by the concurrent subprocess `extendEnsemble` (to update the APPM ensemble). The results show that the former "latency" time scales well w.r.t. the velocity of the input stream (i.e. it is nearly independent of the event rate), and it is far smaller than the average time between consecutive events —namely, 515 msec vs. 87.5 sec when setting $\Theta = 64$ (i.e. at the faster event rate). In other words, not only `APPM-EventProc`

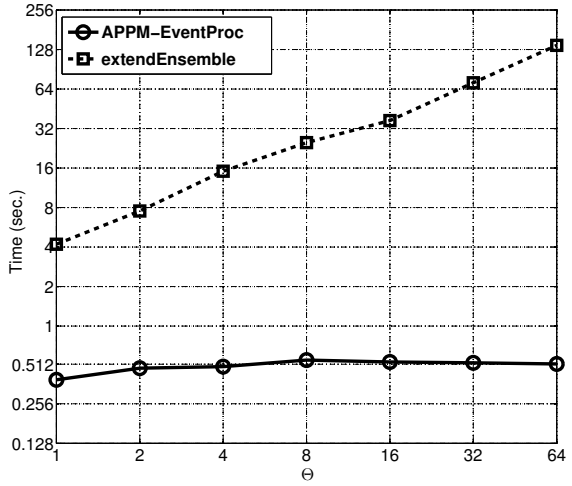


Figure 4: Average times spent to update the APPM ensemble with procedure `extendEnsemble` and to process an event with the main algorithm `APPM-EventProc`. Both axes are in a logarithmic scale. The tests were performed fixing $batchSize\% = 20\%$ and $VP = 20\%$, and simulating different event rates by "duplicating" the events in the log Θ times.

is fast enough to sustain the event stream, but it can also produce updated predictions for the current window in a very timely way —in fact, the latency time is 157,000 times lower than the temporal distance between consecutive checkpoints. Nevertheless, in case a very larger event rate needs to be dealt with, one could distribute the processing of the events among $K > 1$ computation cores (running each its own instantiation of `APPM-EventProc`), in order to reduce the average processing time of about a factor K.

Clearly, the efficiency requirements for `extendEnsemble` are less strict, seeing as the APPM ensemble must be updated only every time a new batch (of $batchSize$ contiguous slots) becomes available. In any case, the ensemble-update time measured in our tests (see Figure 4) is very low even when $\Theta = 64$, compared to the batch duration (namely, 2.3 min vs. about 102 days). Moreover, this time exhibits a good (nearly linear) scalability trend w.r.t. the event rate. Indeed, the time taken to update the model is about 64,000 times lower than the duration of the batch. This means that, whenever a batch terminates, only a negligible fraction of the next batch will be elapsed before the ensemble updating procedure ends. In particular, learning times of the order of some minutes allow us to complete the update within the first slot of the new batch, in time to make more accurate predictions already for the second checkpoint.

6 CONCLUSION

We have proposed a comprehensive event processing framework that allows the aggregate performances of a business process to be monitored in a predictive way, by continuously predicting whether the current window of process instances violates a given A-PPI, based on the stream of event data that process generates. The framework relies on incrementally learning and applying an ensemble of A-PPI prediction models, capable each to integrate both predictions over ongoing process instances and time-series-based predictions for future time slots.

Our proposal advances previous approaches in the literature (in particular, those proposed in [2, 9]) along two main directions: (1) it makes a practical solution for fully exploiting streaming event data (like those that are becoming more and more common in many real-life contexts) in a sufficiently efficient and scalable manner; and (2) it can work in non-stationary process monitoring settings where various kinds of concept drift may occur.

As future work, we plan to investigate the following research lines: (i) monitoring risk-oriented metrics and compliance/security constraints; (ii) dealing with scenarios where the performance outcome of a p-instance may be uncertain even after completing it; (iii) implementing algorithm `APPM-EventProc` in a parallel/distributed way, to make it deal with much larger event production rates.

REFERENCES

[1] K. Chandy and W. Schulte. *Event Processing: Designing IT Systems for Agile Companies.* McGraw-Hill, Inc., New York, NY, USA, 2010.
[2] A. Cuzzocrea, F. Folino, M. Guarascio, and L. Pontieri. Predictive monitoring of temporally-aggregated performance indicators of business processes against low-level streaming events. *Information Systems.* In Press, Accepted Manuscript.
[3] A. Dahanayake, R. J. Welke, and G. Cavalheiro. Improving the understanding of bam technology for real-time decision support. *Int. J. Bus. Inf. Syst.*, 7(1):1–26, 2011.
[4] A. del-Río-Ortega, M. Resinas, C. Cabanillas, and A. R. Cortés. On the definition and design-time analysis of process performance indicators. *Information Systems*, 38(4):470–490, 2013.
[5] R. Elwell and R. Polikar. Incremental learning of concept drift in nonstationary environments. *Trans. Neur. Netw.*, 22(10):1517–1531, 2011.
[6] F. Folino, M. Guarascio, and L. Pontieri. Discovering context-aware models for predicting business process performances. In *Proc. of 20th Int. Conf. on Cooperative Inf. Systems (CoopIS'12)*, pages 287–304, 2012.
[7] F. Folino, M. Guarascio, and L. Pontieri. Discovering high-level performance models for ticket resolution processes. In *On the Move to Meaningful Internet Systems: OTM 2013 Conferences*, pages 275–282, 2013.
[8] F. Folino, M. Guarascio, and L. Pontieri. Mining predictive process models out of low-level multidimensional logs. In *Proc. of 26th Int. Conf. on Advanced Information Systems Engineering (CAISE'14)*, pages 533–547, 2014.
[9] F. Folino, M. Guarascio, and L. Pontieri. A prediction framework for proactively monitoring aggregate process-performance indicators. In *Proc. of 19th IEEE Int. Conf. on Enterprise Distributed Object Computing (EDOC'15)*, pages 128–133, 2015.
[10] S. Grossberg. Nonlinear neural networks: Principles, mechanisms, and architectures. *Neural networks*, 1(1):17–61, 1988.
[11] C. Janiesch, M. Matzner, and O. Müller. A blueprint for event-driven business activity management. In *Proc. of 9th Int. Conf. on Business Process Management (BPM'11)*, pages 17–28, 2011.
[12] M. Kubat, R. Holte, and S. Matwin. Learning when negative examples abound. In *Proc. of European Conf. on Machine Learning (ECML'97)*, pages 146–153, 1997.
[13] A. Leontjeva, R. Conforti, C. D. Francescomarino, M. Dumas, and F. M. Maggi. Complex symbolic sequence encodings for predictive monitoring of business processes. In *Proc. of 13th Int. Conf. on Business Process Management (BPM'15)*, pages 297–313, 2015.
[14] F. M. Maggi, C. Di Francescomarino, M. Dumas, and C. Ghidini. Predictive monitoring of business processes. In *Proc. of 26th Int. Conf. on Advanced Information Systems Engineering (CAiSE'14)*, pages 457–472, 2014.
[15] A. Metzger, P. Leitner, D. Ivanovic, E. Schmieders, R. Franklin, M. Carro, S. Dustdar, and K. Pohl. Comparing and combining predictive business process monitoring techniques. *IEEE Trans. on Systems, Man, and Cybernetics*, 45(2):276–290, 2015.
[16] A. Rogge-Solti and M. Weske. Prediction of remaining service execution time using stochastic petri nets with arbitrary firing delays. In *Proc. of 11th International Conference on Service-Oriented Computing (ICSOC'13)*, pages 389–403, 2013.
[17] W. Schmidt. Business activity monitoring (bam). In *Business Intelligence and Performance Management: Theory, Systems and Industrial Applications*, pages 229–242. Springer London, 2013.
[18] I. Teinemaa, M. Dumas, M. L. Rosa, and F. M. Maggi. Outcome-oriented predictive process monitoring: Review and benchmark. *CoRR*, abs/1707.06766, 2017.
[19] W. M. P. van der Aalst, M. H. Schonenberg, and M. Song. Time prediction based on process mining. *Information Systems*, 36(2):450–475, 2011.